SMALLHOLDER FARMER

PRODUCE MARKETPLACE API

Project Guide & System Requirements Specification

Module: SDT621 – Software Design & Testing
Qualification: OCC Software Engineer Level 6 — SAQA ID 119458
Assessments: Formative 1 (Ch. 1–5) · Formative 2 (Ch. 6–10) · PM-04 Practical

Version 1.0 | April 2026

1. Project Overview

1.1 The Problem

Small-scale (smallholder) farmers in peri-urban and rural South Africa sell produce at low prices to middlemen because they have no direct channel to buyers such as restaurants, spaza shops, schools, and individual households. There is no transparent pricing, no produce availability system, and no order tracking. Farmers lose income; buyers pay inflated prices for the same goods.

1.2 What the System Does

- Farmers register and create produce listings (crop type, quantity, price, harvest date, location).
- Buyers browse, filter, and search listings by location, produce type, and price range.
- Buyers place orders against listings; order status is tracked (Pending → Confirmed → Collected → Cancelled).
- A price history endpoint tracks average prices per produce category over time.
- A simple rating/review system lets buyers rate farmers after collection.

1.3 Assessment Alignment

Assessment	Scope	Due
Formative 1 (50% FA weight)	Chapters 1–5: project scaffold, ProduceListing class, methods, control flow, collections, GET/POST endpoints (Weeks 1–3 foundation)	Week 9 (27 Apr–1 May)
Formative 2 (50% FA weight)	Chapters 6–10: enums, exception handling, structs, interfaces, inheritance, polymorphism, full CRUD + order workflow	Week 10 (4–8 May)
PM-04 Practical (PM-03 mark)	Full project: PA0101 identify tests, PA0102 apply tests, PA0103 produce test report	Week 11 (11–15 May)

1.4 Technology Stack

Component	Technology
Language	C# 12 / .NET 8
Framework	ASP.NET Core Web API (from Week 1)
Data Access	Entity Framework Core — In-Memory DB (MVP); SQLite optional stretch goal
Authentication	JWT Bearer Tokens (introduced Week 3 stretch)
Validation	Data Annotations + FluentValidation (Week 2)
Testing — Manual	Swagger / OpenAPI UI (from Week 1)
Testing — Auto	MSTest unit + integration tests (Week 4–5, PA0102)
Documentation	Swagger / OpenAPI (built-in, visible from Day 1)
IDE	Visual Studio 2022

2. Domain Model

The following entities form the MVP. Each maps to a C# class introduced progressively across 5 weeks.

2.1 Farmer

Property	Type	Description
FarmerId	int	Primary key (auto-incremented)
FullName	string	Farmer's full name
Email	string	Unique contact email
PhoneNumber	string	Contact number
FarmName	string	Trading name of the farm
Location	string	Town / suburb
Province	Province (enum)	Province enum (Week 2)
Rating	double	Average rating from buyer reviews (0.0–5.0)
IsVerified	bool	Whether the farmer is marketplace-verified

2.2 ProduceListing

Property	Type	Description
ListingId	int	Primary key
FarmerId	int	FK to Farmer
ProductName	string	Name of the crop / produce (min 3 chars)
Category	Category (enum)	Vegetables | Fruit | Grain | Dairy | Other
PricePerKg	double	Asking price per kilogram (> 0)
QuantityKg	double	Available quantity in kg (> 0)
IsAvailable	bool	Whether the listing is currently active
HarvestDate	DateTime	Date produce was or will be harvested
DateListed	DateTime	When the listing was created
Description	string?	Optional extra details

2.3 Buyer

Property	Type	Description
BuyerId	int	Primary key

Property	Type	Description
FullName	string	Buyer's full name
Email	string	Unique contact email
PhoneNumber	string	Contact number
BuyerType	BuyerType (enum)	Individual | SpazaShop | Restaurant | School | Other
Location	string	Delivery / pickup location

2.4 Order

Property	Type	Description
OrderId	int	Primary key
BuyerId	int	FK to Buyer
ListingId	int	FK to ProduceListing
QuantityOrdered	double	Kg ordered (must not exceed listing quantity)
TotalPrice	double	QuantityOrdered × PricePerKg (calculated)
Status	OrderStatus (enum)	Pending | Confirmed | Collected | Cancelled
OrderDate	DateTime	When order was placed
CollectionDate	DateTime?	Nullable — set when collected
Notes	string?	Optional buyer notes

2.5 Review

Property	Type	Description
ReviewId	int	Primary key
BuyerId	int	FK to Buyer
FarmerId	int	FK to Farmer
OrderId	int	FK to Order (one review per order)
Rating	int	1–5 star rating
Comment	string?	Optional text comment
DatePosted	DateTime	When the review was submitted

2.6 Enums

Enum	Values
Category	Vegetables = 0, Fruit = 1, Grain = 2, Dairy = 3, Other = 4
OrderStatus	Pending = 0, Confirmed = 1, Collected = 2, Cancelled = 3
BuyerType	Individual = 0, SpazaShop = 1, Restaurant = 2, School = 3, Other = 4
Province	Gauteng = 0, WesternCape = 1, KwaZuluNatal = 2, Limpopo = 3, Mpumalanga = 4, NorthWest = 5, FreeState = 6, NorthernCape = 7, EasternCape = 8

3. Weekly Feature & Activity Breakdown

Each week's class activities directly build the MVP. The Swagger UI is used for manual testing from Day 1. PM-04 test cases are collected throughout and formalised into the test report in Week 5.

Week 1 — C# Foundations + API Scaffold (13–15 April)

SDT621 Chapters: 1 & 2 (Mon) · 3 & 4 (Tue) · 5 & 6 (Wed)

Schedule: Mon 2 hrs · Tue 3 hrs · Wed 2 hrs = 7 hrs contact time

Features Built

Day	Feature / Deliverable	Class / File	C# Concept
Mon	ASP.NET Core Web API project scaffolded and running in Swagger	Program.cs	Project structure, namespaces (Ch. 1)
Mon	Domain variables identified and declared	Program.cs	Data types, operators, var (Ch. 2)
Tue	ProduceListing model class	Models/ProduceListing.cs	Classes, properties, constructors (Ch. 4)
Tue	GetFormattedSummary() method	ProduceListing.cs	Methods, return types (Ch. 3)
Tue	CalculateRevenue() method	ProduceListing.cs	Arithmetic, method overloading (Ch. 3)
Tue	GET /api/produce (single hardcoded listing)	Controllers/ProduceController.cs	Controller, [ApiController], routing (Ch. 4)
Tue	GET /api/produce/summary (formatted string)	ProduceController.cs	Return types, attributes
Wed	Static in-memory List<ProduceListing>	ProduceController.cs	List<T>, collections (Ch. 6)
Wed	POST /api/produce (add listing to list)	ProduceController.cs	HTTP POST, model binding
Wed	GET /api/produce/available (filter by IsAvailable)	ProduceController.cs	foreach, if/else (Ch. 5)
Wed	GET /api/produce/category/{category}	ProduceController.cs	switch, route params (Ch. 5)
Wed	GetCategoryLabel() helper method with switch	ProduceListing.cs	switch statement

PM-04 Test Activities — Week 1 (PA0101: Identify)

TC#	Test Case (Given / When / Then)	Endpoint / Method
TC-01	Given the API is running, when GET /api/produce is called, then a ProduceListing object is returned with all required fields.	GET /api/produce
TC-02	Given a ProduceListing with PricePerKg=10 and QuantityKg=5, when CalculateRevenue() is called, then the result should be 50.	CalculateRevenue()

TC#	Test Case (Given / When / Then)	Endpoint / Method
TC-03	Given 3 listings are POSTed, when GET /api/produce is called, then a JSON array of 3 listings is returned.	GET /api/produce
TC-04	Given 3 listings where 2 have IsAvailable=true, when GET /api/produce/available is called, then exactly 2 listings are returned.	GET /api/produce/available
TC-05	Given listings of mixed categories, when GET /api/produce/category/Fruit is called, then only Fruit listings are returned.	GET /api/produce/category/{category}

Formative 1 Contribution (Week 1)

- ProduceListing class with all properties — correct types, access modifiers.
- GetFormattedSummary() and CalculateRevenue() methods demonstrating return types and parameters.
- GET /api/produce, POST /api/produce, GET /api/produce/available endpoints working in Swagger.
- Test cases TC-01 to TC-05 documented in Test Cases document.

Week 2 — Exception Handling, Enums, Structs, Repository Layer (20–24 April)

SDT621 Chapters: 7 & 8 — Exception handling · Enums · Structs

Schedule: Mon 2 hrs · Tue 3 hrs · Wed 2 hrs = 7 hrs contact time | Week 14 calendar (FA1/FA2/PM handed out)

Features Built

Day	Feature / Deliverable	Class / File	C# Concept
Mon	Category, OrderStatus, BuyerType, Province enums	Models/Enums/	Enums (Ch. 8)
Mon	Refactor ProduceListing to use Category enum	ProduceListing.cs	Enum fields, switch on enum
Mon	FarmerLocation struct (FarmName, Town, Province)	Models/FarmerLocation.cs	Structs (Ch. 8)
Tue	ProduceRepository class (Add, GetAll, GetById, GetByCategory, GetAvailable)	Repositories/ProduceRepository.cs	Collections, loops, encapsulation (Ch. 6)
Tue	Replace static controller list with ProduceRepository	ProduceController.cs	Dependency (manual for now)
Tue	Input validation: reject PricePerKg ≤ 0 or empty ProductName	ProduceController.cs / DTOs	if/else guards
Tue	try/catch in POST /api/produce for FormatException	ProduceController.cs	Exception handling (Ch. 7)
Tue	Custom ListingNotFoundException class	Exceptions/ListingNotFoundException.cs	Custom exceptions (Ch. 7)
Wed	GET /api/produce/{id} — throws ListingNotFoundException if not found	ProduceController.cs	try/catch, custom exception

Day	Feature / Deliverable	Class / File	C# Concept
Wed	Farmer model class + FarmerRepository stub	Models/Farmer.cs	Classes, encapsulation (Ch. 4)
Wed	POST /api/farmers endpoint	Controllers/FarmerController.cs	HTTP POST, model binding

PM-04 Test Activities — Week 2 (PA0101 continued)

TC#	Test Case (Given / When / Then)	Endpoint / Method
TC-06	Given a valid ListingId, when GET /api/produce/{id} is called, then 200 OK with the correct listing is returned.	GET /api/produce/{id}
TC-07	Given an invalid ListingId (e.g. 9999), when GET /api/produce/{id} is called, then 404 Not Found is returned.	GET /api/produce/{id}
TC-08	Given a POST body with PricePerKg=0, when POST /api/produce is called, then 400 Bad Request is returned.	POST /api/produce
TC-09	Given a POST body with an empty ProductName, when POST /api/produce is called, then 400 Bad Request is returned.	POST /api/produce
TC-10	Given the repository has 4 listings in category Grain, when GetByCategory(Category.Grain) is called, then 4 listings are returned.	ProduceRepository.GetByCategory()
TC-11	Given a valid Farmer body, when POST /api/farmers is called, then 201 Created with FarmerId is returned.	POST /api/farmers

Formative 1 Contribution (Week 2 — due Week 9)

- All 4 enums implemented and used in ProduceListing.
- FarmerLocation struct with Province enum.
- ProduceRepository with Add, GetAll, GetById, GetByCategory, GetAvailable methods.
- Exception handling: custom ListingNotFoundException, try/catch in GET /{id}, validation guards in POST.
- Test cases TC-06 to TC-11 documented.

Week 3 — Interfaces, Inheritance & Polymorphism (9–13 May)

SDT621 Chapters: 9 & 10 — Interfaces · Inheritance · Polymorphism

Schedule: Mon 2 hrs · Tue 3 hrs · Wed 2 hrs = 7 hrs contact time | Week 16 calendar (FA1 due)

Features Built

Day	Feature / Deliverable	Class / File	C# Concept
Mon	IRepository<T> generic interface (Add, GetAll, GetById, Delete)	Interfaces/IRepository.cs	Interfaces (Ch. 9)
Mon	ProduceRepository implements IRepository<ProduceListing>	ProduceRepository.cs	Interface implementation
Mon	FarmerRepository implements IRepository<Farmer>	FarmerRepository.cs	Interface reuse

Day	Feature / Deliverable	Class / File	C# Concept
Tue	Person base class (FullName, Email, PhoneNumber, GetContactInfo())	Models/Person.cs	Base class, inheritance (Ch. 10)
Tue	Farmer : Person (adds FarmName, Location, Rating, IsVerified)	Models/Farmer.cs	Derived class, override
Tue	Buyer : Person (adds BuyerType, Location)	Models/Buyer.cs	Derived class
Tue	Polymorphic GetContactInfo() — different output for Farmer vs Buyer	Person, Farmer, Buyer	virtual / override polymorphism (Ch. 10)
Tue	Order model + OrderRepository	Models/Order.cs	Classes, encapsulation
Wed	POST /api/orders — place an order against a listing	Controllers/OrderController.cs	HTTP POST, business logic
Wed	PATCH /api/orders/{id}/confirm — confirm order	OrderController.cs	PATCH, status workflow
Wed	PATCH /api/orders/{id}/collect — mark collected, reduce listing quantity	OrderController.cs	State machine, quantity update
Wed	INotifiable interface — EmailNotifier and SmsNotifier stubs	Notifications/	Interface + polymorphism

PM-04 Test Activities — Week 3 (PA0101 continued)

TC#	Test Case (Given / When / Then)	Endpoint / Method
TC-12	Given a valid listing and buyer, when POST /api/orders is called, then 201 Created is returned with Status = Pending.	POST /api/orders
TC-13	Given an order quantity exceeding listing quantity, when POST /api/orders is called, then 422 Unprocessable Entity is returned.	POST /api/orders
TC-14	Given a Pending order, when PATCH /api/orders/{id}/confirm is called, then Status changes to Confirmed.	PATCH /orders/confirm
TC-15	Given a Confirmed order, when PATCH /api/orders/{id}/collect is called, then Status = Collected and listing QuantityKg is reduced.	PATCH /orders/collect
TC-16	When GetContactInfo() is called on a Farmer object, the output includes the FarmName. When called on a Buyer object, it includes the BuyerType.	GetContactInfo() polymorphism

Formative 2 Contribution (Week 3 — due Week 10)

- IRepository<T> interface implemented by ProduceRepository and FarmerRepository.
- Person → Farmer and Person → Buyer inheritance hierarchy with polymorphic GetContactInfo().
- Order model, OrderRepository, and full order workflow endpoints (POST, confirm, collect).
- Test cases TC-12 to TC-16 documented.

Week 4 — Full CRUD, Price History, Reviews & Error Middleware (16–20 May)

SDT621: Revision Chapters 1–10 + API consolidation

Schedule: Mon 2 hrs · Tue 3 hrs · Wed 2 hrs = 7 hrs | Week 17 calendar (FA2 due)

Features Built

Day	Feature / Deliverable	Class / File	C# Concept
Mon	PUT /api/produce/{id} — update listing (price, quantity)	ProduceController.cs	HTTP PUT, validation
Mon	DELETE /api/produce/{id} — soft-delete (set IsAvailable=false)	ProduceController.cs	HTTP DELETE, soft delete pattern
Mon	GET /api/produce?minPrice=x&maxPrice=y — price range filter	ProduceController.cs	Query parameters, LINQ-style filter
Tue	Review model + ReviewRepository + POST /api/reviews	Reviews/	Classes, validation
Tue	GET /api/farmers/{id}/reviews — all reviews for a farmer	FarmerController.cs	FK queries, filtering
Tue	PATCH /api/farmers/{id}/rating — recalculate average rating	FarmerController.cs	Aggregation logic
Tue	PriceHistory record per listing (store price changes on update)	Models/PriceHistory.cs	Value objects, list tracking
Tue	GET /api/produce/{id}/price-history	ProduceController.cs	Collection return
Wed	Global exception handling middleware (catch all unhandled → 500 with message)	Middleware/ErrorHandlingMiddleware.cs	Middleware, exception types
Wed	GET /api/dashboard/summary (total listings, active, by category, avg price)	DashboardController.cs	Aggregation, LINQ-style grouping
Wed	Swagger XML comments on all endpoints (/// <summary>)	All controllers	Documentation

PM-04 Test Activities — Week 4 (PA0102: Apply Tests)

TC#	Test (Manual — Swagger / Postman)	Endpoint
TC-17	Given a listing with PricePerKg=20, when PUT /api/produce/{id} updates price to 25, then GET /{id} returns 25.	PUT /api/produce/{id}
TC-18	When DELETE /api/produce/{id} is called, then GET /{id} returns IsAvailable=false (not actually deleted).	DELETE /api/produce/{id}
TC-19	Given listings at R10, R20, R30/kg, when GET /api/produce?minPrice=15&maxPrice=25, then only the R20 listing is returned.	GET with price filter
TC-20	Given a completed Order, when POST /api/reviews is called with Rating=4, then 201 Created is returned.	POST /api/reviews
TC-21	After 3 reviews with ratings 3, 4, 5 for one farmer, when PATCH /api/farmers/{id}/rating is called, the farmer's Rating becomes 4.0.	PATCH /api/farmers/{id}/rating
TC-22	When GET /api/dashboard/summary is called, the TotalListings count matches the number of POSTed listings.	GET /api/dashboard/summary

Formative 2 Contribution (Week 4 — due Week 10)

- Full CRUD for ProduceListing (GET, POST, PUT, DELETE with soft-delete).
- Review system — POST, GET by farmer, rating recalculation.
- Price history tracking and retrieval endpoint.
- Dashboard summary endpoint with aggregations.
- Global error handling middleware — no unhandled 500s.
- Test cases TC-17 to TC-22 applied and results recorded.

Week 5 — MSTest Testing, EF Core (optional), PM-04 Test Report (23–27 May)

SDT621: Summative exam prep + PM-04 PA0102 automated tests + PA0103 test report

Schedule: Mon 2 hrs · Tue 3 hrs · Wed 2 hrs = 7 hrs | Week 18 calendar (PM-04 due)

Features Built

Day	Feature / Deliverable	Class / File	C# Concept
Mon	MSTest test project scaffold	FarmerMarket.Tests/	Test project, NuGet (MSTest.TestFramework, MSTest.TestAdapter)
Mon	Unit tests: ProduceRepository (Add, GetById valid, GetById invalid)	ProduceRepositoryTests.cs	MSTest [TestClass], [TestMethod], Assert
Mon	Unit tests: CalculateRevenue() and GetCategoryLabel()	ProduceListingTests.cs	Unit test, edge cases
Tue	Unit tests: Order quantity validation (exceed listing qty)	OrderTests.cs	Exception assertions
Tue	Integration tests: POST /api/produce happy path → 201	ProduceControllerTests.cs	MSTest + HttpClient (TestServer)
Tue	Integration tests: POST /api/produce invalid body → 400	ProduceControllerTests.cs	HTTP integration, Assert.AreEqual
Tue	Integration tests: GET /api/produce/{id} invalid → 404	ProduceControllerTests.cs	404 integration, Assert.AreEqual
Wed	EF Core In-Memory DB wired up (replace static lists — stretch goal)	Program.cs / DbContext	EF Core (extension)
Wed	Complete PM-04 Test Report (PA0103)	TestReport.docx	PM-04 submission evidence
Wed	Final Swagger walkthrough — all endpoints documented and tested	All controllers	OpenAPI review

PM-04 Test Activities — Week 5 (PA0102: MSTest Automated)

TC#	Automated Test Method	Class
TC-23	[TestMethod] Add_NewListing_IncreasesCountByOne()	ProduceRepositoryTests
TC-24	[TestMethod] GetById_ValidId_ReturnsCorrectListing()	ProduceRepositoryTests

TC#	Automated Test Method	Class
TC-25	[TestMethod] GetById_InvalidId_ThrowsListingNotFoundException()	ProduceRepositoryTests
TC-26	[TestMethod] CalculateRevenue_Returns_PriceTimesQuantity()	ProduceListingTests
TC-27	[TestMethod] GetCategoryLabel_Fruit_ReturnsCorrectLabel()	ProduceListingTests
TC-28	[TestMethod] PlaceOrder_ExceedsQuantity_ThrowsValidationException()	OrderTests
TC-29	[TestMethod] PostListing_ValidBody_Returns201Created()	ProduceControllerTests
TC-30	[TestMethod] PostListing_MissingProductName_Returns400BadRequest()	ProduceControllerTests
TC-31	[TestMethod] GetListing_InvalidId_Returns404NotFound()	ProduceControllerTests
TC-32	[TestMethod] GetDashboardSummary_Returns_CorrectTotalCount()	DashboardControllerTests

4. System Requirements Specification (SRS)

4.1 Functional Requirements

Functional requirements describe what the system must do (PM-04 IAC0101: functional tests are identified).

FR-01: Produce Listings

ID	Requirement	Acceptance Criteria
FR-01.1	A farmer shall be able to create a produce listing with ProductName, Category, PricePerKg, QuantityKg, HarvestDate.	POST /api/produce returns 201 with ListingId.
FR-01.2	The system shall reject listings with PricePerKg ≤ 0 or QuantityKg ≤ 0.	POST returns 400 Bad Request.
FR-01.3	The system shall reject listings with a ProductName shorter than 3 characters.	POST returns 400 Bad Request.
FR-01.4	A buyer shall be able to retrieve all available listings.	GET /api/produce/available returns only IsAvailable=true listings.
FR-01.5	A buyer shall be able to filter listings by category.	GET /api/produce/category/{category} returns correct subset.
FR-01.6	A buyer shall be able to filter listings by price range.	GET /api/produce?minPrice=x&maxPrice=y returns correct subset.
FR-01.7	A farmer shall be able to update listing price and quantity.	PUT /api/produce/{id} updates values; GET /{id} reflects changes.
FR-01.8	Deleting a listing shall mark it unavailable, not physically delete it.	DELETE /api/produce/{id} sets IsAvailable=false.

FR-02: Orders

ID	Requirement	Acceptance Criteria
FR-02.1	A buyer shall be able to place an order against a listing.	POST /api/orders returns 201 with Status=Pending.
FR-02.2	The system shall reject orders where QuantityOrdered exceeds listing QuantityKg.	POST /api/orders returns 422 Unprocessable Entity.
FR-02.3	A farmer shall be able to confirm an order.	PATCH /api/orders/{id}/confirm changes Status to Confirmed.
FR-02.4	On collection, listing QuantityKg shall be reduced by QuantityOrdered.	PATCH /api/orders/{id}/collect reduces listing quantity.
FR-02.5	Either party shall be able to cancel a Pending or Confirmed order.	PATCH /api/orders/{id}/cancel changes Status to Cancelled.

FR-03: Farmers & Buyers

ID	Requirement	Acceptance Criteria
FR-03.1	A farmer shall be able to register with FullName, Email, FarmName, Location, Province.	POST /api/farmers returns 201 with FarmerId.
FR-03.2	A buyer shall be able to register with FullName, Email, BuyerType, Location.	POST /api/buyers returns 201 with BuyerId.
FR-03.3	Duplicate emails shall be rejected for both farmers and buyers.	POST returns 409 Conflict.

FR-04: Reviews & Ratings

ID	Requirement	Acceptance Criteria
FR-04.1	A buyer shall be able to post a review for a farmer after a Collected order.	POST /api/reviews returns 201.
FR-04.2	Rating must be between 1 and 5.	POST /api/reviews with Rating=6 returns 400.
FR-04.3	A farmer's average rating shall be recalculated when a new review is posted.	PATCH /api/farmers/{id}/rating returns updated rating.

FR-05: Dashboard

ID	Requirement	Acceptance Criteria
FR-05.1	The system shall provide a summary of total listings, active listings, listings by category, and average price per category.	GET /api/dashboard/summary returns accurate aggregated data.

4.2 Non-Functional Requirements

Non-functional requirements describe quality attributes (PM-04 IAC0101: non-functional tests identified).

ID	Category	Requirement	Test Approach
NFR-01	Performance	GET /api/produce must respond within 500 ms for up to 200 listings.	Time the response in Swagger / Postman.
NFR-02	Reliability	The API must never return an unhandled 500 error — all exceptions return structured JSON.	Send malformed JSON and missing fields — verify no raw 500s.
NFR-03	Usability	All endpoints must be documented in Swagger with XML summary comments.	Open /swagger — verify all routes have descriptions.
NFR-04	Maintainability	No business logic in controllers — all logic in repository / service classes.	Code review checklist during PM-04 assessment.
NFR-05	Data Integrity	Quantity ordered must never exceed listing quantity available.	POST /api/orders with excess quantity must return 422.

5. PM-04 Test Report Template (PA0103)

Complete this report at the end of Week 5. This is your PM-04 logbook evidence for IAC0103: A test report is produced.

Learner Name:	
Student Number:	
Date of Testing:	
System Under Test:	Smallholder Farmer Produce Marketplace API v1.0
Testing Methods:	Manual (Swagger UI) — Weeks 1–4 | Automated (MSTest) — Week 5
Environment:	Visual Studio 2022 · .NET 8 · ASP.NET Core Web API · EF Core In-Memory
Mentor Name:	

5.1 Test Execution Results

TC#	Test Case Description	Type	Expected Result	Actual Result	Pass / Fail
TC-01	GET /api/produce returns ProduceListing with all fields	Functional	200 OK + object		
TC-02	CalculateRevenue(10, 5) returns 50	Functional	50		
TC-03	3 POSTed listings → GET returns array of 3	Functional	Count = 3		
TC-04	2 available of 3 → GET /available returns 2	Functional	Count = 2		
TC-05	GET /category/Fruit returns only Fruit	Functional	Fruit only		
TC-06	GET /{id} valid → 200 correct listing	Functional	200 OK		
TC-07	GET /{id} invalid → 404	Functional	404 Not Found		
TC-08	POST price=0 → 400	Functional	400 Bad Request		
TC-12	POST /orders valid → 201 Pending	Functional	201 Created		
TC-13	POST /orders qty > listing → 422	Functional	422 Unprocessable		
TC-15	PATCH collect → qty reduced	Functional	Qty reduced		

TC#	Test Case Description	Type	Expected Result	Actual Result	Pass / Fail
TC-19	GET price filter → correct subset	Functional	Correct listings		
TC-21	3 reviews → avg rating = 4.0	Functional	4.0		
TC-22	Dashboard total matches POSTed count	Functional	Correct count		
TC-29	MSTest: POST valid → 201 (automated)	Functional	201 Created		
TC-30	MSTest: POST no name → 400 (automated)	Functional	400 Bad Request		
NFR-01	GET /api/produce responds < 500 ms	Non-Functional	< 500 ms		
NFR-02	Malformed JSON → structured 400, not 500	Non-Functional	400 Bad Request		
NFR-03	Swagger lists all endpoints with summaries	Non-Functional	All routes visible		

5.2 Defects Found

Defect #	Description	Steps to Reproduce	Severity	Status
D-01				
D-02				

5.3 Test Summary

Metric	Value
Total Test Cases Executed	
Tests Passed	
Tests Failed	
Defects Identified	
Automated Tests (MSTest)	
Manual Tests (Swagger)	
Overall Result	

5.4 Conclusion

(Write 3–5 sentences: Was the system functional? What was tested and what was not? Any defects still open? Recommendations for next iteration.)

Learner Signature:	**Mentor Signature:**
_____ Date: _____	_____ Date: _____